[ EXECUTIVE TECH WHITEPAPER · METADATA SANITIZED ]

# AI 에이전트를 위한 2nd Brain과 Eval 프레임워크 아키텍처

분석 대상: 메타 시니어 엔지니어 하재상(실밸개발자) 77분 풀 세미나 전수 분석 | 적용 분야: 모바일 통신/물리계층(PHY) 임베디드 SW

[핵심 요약 (Executive Summary)]
본 백서는 실리콘밸리 메타 시니어 엔지니어 하재상님의 프로덕션 세미나를 전수 분석하여, 자율 코딩 에이전트 도입의 핵심 성공 요소인 '컨텍스트 계층(2nd Brain = Codebase Graph + Code Memory)'과 '검증 계층(Eval = Deterministic Hard Check + K-Judge Majority Voting + Sandbox + CI Gate)'의 엔터프라이즈 아키텍처를 규명한다. 또한 단순 프롬프팅에서 시스템 하네스로의 패러다임 전환과 모바일 통신 물리계층(PHY) C/C++ 펌웨어 리뷰 파이프라인으로의 1:1 적용 방안을 제시한다.

## 1. AI 코딩 패러다임 전환: 코드 생성에서 '검증 병목'으로

과거 개발의 병목이 코드 타이핑과 기본 구현이었다면, LLM 도입 이후의 진짜 병목은 '쏟아지는 거대 PR을 과연 믿고 머지할 수 있는가'라는 검증(Verification) 영역으로 전이되었다. 메타 하재상 엔지니어는 사내 대시보드 기준 월간 $30,000 상당의 에이전트 API를 집중 사용하면서, 하네스(규칙, 툴, 프롬프트)를 아무리 정교하게 세팅해도 자동화된 검증(Eval) 체계가 없으면 인간이 일일이 대조하느라 오히려 피로도와 인지 부하가 가중된다는 점을 실증하였다.

## 2. [메타 프로덕션 실전 사례] 성능 프로파일링 10시간 → 30분 단축

메타 대규모 분산 시스템에서 특정 서비스의 응답 지연(Latency) 급증 원인을 밝히는 작업은 기존 시니어 엔지니어가 Flame Graph와 Trace 데이터를 수작업으로 대조하며 평균 10시간 이상 소요되던 고난도 작업이었다. 이를 해결하기 위해 메타 팀은 다음 3가지 요소를 결합하였다:

- 표준화 도구 (Profiling MCP): Flame Graph 메트릭을 LLM이 직접 수치 질의할 수 있는 표준 API 제공
- 시니어 도메인 지식 (Skill Manual): 메타 시니어들이 병목을 진단할 때 쓰는 5단계 체크리스트와 가설 검증 룰을 마크다운 화
- 코드베이스 지도 (2nd Brain): 함수 간 호출 관계를 즉시 파악하여 후보군을 좁히고 Blast Radius 계산

결과: 10시간의 수작업 트리아지가 30분 만에 원인 규명 및 패치 제안까지 완료됨.

## 3. 2nd Brain 아키텍처: Codebase Graph + Code Memory

에이전트에게 프로젝트 맥락을 매 세션 주입하기 위한 2대 축은 구조적 지도(Codebase Graph)와 역사적 맥락(Code Memory)이다.

| 구분 | Grep / 단순 텍스트 검색 | Codebase Graph (AST Compiler) | Code Memory (Why 히스토리) |
|---|---|---|---|
| **인덱싱 방식** | 정규표현식 단순 문자열 매칭 | TS/Clang AST 파서, Symbol ID 부여 | Git 커밋, PR 토론, ADR, BM25 엔진 |
| **정확도 / 노이즈** | 주석/더미 대량 유입 (노이즈 90%) | 결정론적 100% 호출/정의 매핑 | 희귀 전문 용어 중심 의사결정 추적 |
| **파급 반경 (Blast)** | 추적 불가 (매번 누락 발생) | BFS Hopping으로 2~3차 파급 정밀 계산 | 과거 버그 회피 사유(Why) 즉시 매핑 |
| **토큰 / 레이턴시** | 무관 파일 과다 주입으로 낭비 심각 | 핵심 의존성만 컴팩트 주입 (효율 극대) | 라우팅 룰로 필요 시에만 선택적 조회 |

트레이드오프 및 2원화 라우팅 정책 (Routing Rule): 모든 작업에 Brain을 태우면 레이턴시와 오버헤드가 발생한다. 단일 파일의 국소 수정은 Fast Path(직접 LLM 수정)로 우회하고, 인터페이스 변경 및 공용 모듈 리팩토링 시에만 Brain Path(Graph BFS + Why 메모리 선행 조회)를 강제해야 한다.

## 4. Eval 체계: 결정론적 Hard Check + K-Judge Majority Voting

Anthropic의 Claude Code 시스템 프롬프트 변경 참사나 Amazon, Vercel의 사례처럼, 정량적 평가 없는 에이전트 스킬 배포는 치명적인 프로덕션 장애를 부른다. 메타의 2단계 Eval 구조는 다음과 같다:

1. Stage 1: Hard Check (결정론적 게이트)
컴파일(Clang exit code == 0), 린트(Clang-Tidy), 단위 테스트, 입출력 JSON 스키마를 코드로 검증한다. 단 1개라도 불합격 시 즉시 0점 탈락(Short-circuit) 처리하여 고비용 LLM 심사를 생략한다.
2. Stage 2: LLM-as-a-Judge with K=3 Majority Voting (정성 다수결 투표)
하드 체크를 통과한 패치에 대해 아키텍처 규칙 준수와 예외 처리를 검증한다. LLM의 확률적 분산(Variance)을 제거하기 위해 K=3회 독립 실행(3인의 심사위원 다수결)을 거쳐 2표 이상 찬성 시에만 합격 처리한다.
3. 무균 샌드박스 격리 (Sandbox Isolation):
에이전트가 정답(Ground Truth)이나 로컬 전역 설정(~/.claude/settings)을 오염시키지 않도록 독립된 임시 컨테이너에서 무균 실행한다.
4. CI Gatekeeper:
'CI에서 자동 실행되지 않는 Eval은 없는 것과 같다.' 모든 PR 머지 전 Eval 통과를 GitHub Actions로 강제한다.

## 5. 메타의 Eval 성숙도 5단계 및 스킬 개발의 3대 필수 질문

- Level 0 (느낌 / Vibe): '어? 잘 되는 것 같은데?' 감에 의존하는 가장 위험한 단계.
- Level 1 (단위 테스트 & 린트): 기본적인 하드 체크 자동화 구축.
- Level 2 (오프라인 Eval): 실무 전형 시나리오 20~30개 엄선 골든 프롬프트(Golden Prompts) 정량 점수화.
- Level 3 (CI Gatekeeper): PR 머지 전 자동 Eval 통과 강제.
- Level 4 (Self-Improving Loop): 실패 케이스 피드백을 에이전트가 분석하여 프롬프트/스킬 자체 자동 개선.

스킬 개발 시 반드시 답해야 하는 3대 질문:
① Triggering Accuracy: 불려야 할 때 불리고, 안 불려야 할 때 안 불리는가?
② Performance Gain: Baseline 대비 실질적인 성능/성공률 향상이 있는가?
③ Non-Regression: 새로운 스킬 추가로 인해 기존 기능이 깨지지 않는가?

## 6. 모바일 통신 물리계층(PHY) SW 그룹 2nd Brain 적용 청사진

본 세미나의 아키텍처를 우리 그룹(Mobile SW 통신 프로토콜 및 물리계층 임베디드 SW)에 적용하는 구체적 청사진은 다음과 같다:

| 계층 | 메타 범용 엔지니어링 패턴 | 모바일 PHY / 임베디드 SW 적용 모델 |
|---|---|---|
| **Code Graph** | TS/Python AST 기반 함수 호출 매핑 | ARM Clang AST 기반 하드웨어 레지스터 맵(MMIO Address, Bitfield), ISR 인터럽트 벡터, 3GPP FSM 상태 머신 링킹 |
| **Code Memory** | 일반 PR 토론 및 아키텍처 결정(ADR) | 3GPP 표준 릴리즈 변경점, 퀄컴/경쟁사 칩셋 호환성, 실리콘 에라타(Silicon Errata) 및 과거 IODT 필드 이슈 이력 BM25 인덱싱 |
| **Firmware Eval** | 웹 서비스 린트 및 단위 테스트 | Stage 1: 타겟 크로스 컴파일러 빌드 + I/D-Cache 사용량 검증<br>Stage 2: ISR 동시성 경합 및 us 실시간 데드라인 규격 다수결 판정 |

## 7. 결론 및 엔지니어 철학: 에이전트 시대, 인간의 주도권

"에이전트를 돌리느라 24시간 쉬지 못하고 건강과 가족을 해치는 것은 도구의 본질을 전도시키는 것입니다. 우리가 왜 AI를 씁니까? 에이전트에게 주도권을 뺏기지 마세요. 나의 1번 가치(가족, 건강, 엔지니어링의 본질)를 확고히 세우고, 에이전트는 나를 그 가치로 데려다주는 강력한 지렛대로 부려야 합니다."

최종 시사점: 설계와 전략, 가치 판단은 인간 테크리드가 확고히 쥐고, 구현과 검증은 견고한 시스템 하네스(2nd Brain + Eval

CI Gate)에 위임하는 것이 에이전틱 엔지니어링의 정답이다.